bicsa

DOCUMENTACIÓN OFICIAL DE INGENIERÍA

Manual Técnico de Arquitectura, Despliegue y Operaciones

Guía integral de infraestructura, componentes, contenedores Docker, bases de datos PostgreSQL, configuración DNS, certificados SSL y procedimientos de mantenimiento para el Sistema de Gestión Comercial BICSA.

SISTEMA	VERSIÓN DE RELEASE
BICSA Gestión Comercial	**V1.3 (Producción)**
ENTORNO DE PRODUCCIÓN	CLASIFICACIÓN
Oracle Cloud Infrastructure (OCI)	**Documento Técnico Confidencial**
FECHA DE ACTUALIZACIÓN	ÁREA RESPONSABLE
Octubre 2026	**Ingeniería de Software e Infraestructura**

© 2026 BICSA S.A. Todos los derechos reservados.

Asunción, Paraguay

Índice General de Contenidos

1. Ficha Técnica y Resumen del Proyecto 3
2. Arquitectura del Sistema y Diagrama de Red 3
3. Stack Tecnológico Detallado 4
4. Composición del Despliegue Docker Compose 5
5. Variables de Entorno y Configuración (.env) 6
6. Modelo de Base de Datos PostgreSQL 16 7
7. Infraestructura en la Nube (Oracle Cloud OCI) 9
8. Configuración de Dominio y Registros DNS 10
9. Certificados de Seguridad SSL/TLS y Terminación HTTPS 11
10. Mecanismos de Autenticación, JWT y Auto-Cierre a Medianoche 12
11. Estrategia de Copias de Seguridad (Backups y Restauración) 13
12. Comandos Operativos de Despliegue y Mantenimiento 14

1. Ficha Técnica y Resumen del Proyecto

El **Sistema de Gestión Comercial BICSA** es una plataforma de clase empresarial diseñada para digitalizar, supervisar y auditar la totalidad de las operaciones comerciales de campo y reuniones corporativas llevadas a cabo por los ejecutivos de cuenta. Su objetivo principal es erradicar los registros informales en papel y planillas desarticuladas, proporcionando trazabilidad geográfica, auditoría de odómetro vehicular, registro de traslados en plataformas digitales, prorrateo de costos de movilidad e indicadores ejecutivos en tiempo real.

Atributo	Especificación Técnica
Nombre Oficial	Sistema Comercial BICSA (Gestión Comercial)
Versión de Producción	Release V1.3.0
Tipo de Aplicación	Single Page Application (SPA) + Progressive Web App (PWA) instalable + REST API
Paradigma de Arquitectura	Arquitectura contenerizada multi-servicio en 3 capas desacopladas
Plataforma de Host	Oracle Cloud Infrastructure (OCI) - Máquina Virtual Ubuntu Linux
Control de Versiones	Git / Repositorio GitHub: `Matiasmart9/SISTEMA_COMERCIAL_BICSA` (rama `main`)
Orquestador Local	Docker Engine 26+ y Docker Compose V2

2. Arquitectura del Sistema y Diagrama de Red

El sistema opera bajo un esquema de **4 contenedores Docker** independientes interconectados a través de una red virtual interna con bridge aislado (`bicsa_net`), asegurando que los servicios críticos (como PostgreSQL y el Backend Node.js) no queden expuestos directamente a internet, sino únicamente a través del Proxy Inverso seguro Nginx.



1. **Ingreso del Cliente:** El navegador o smartphone envía peticiones HTTP/HTTPS al dominio público.
2. **Ruteo Nginx:** Las rutas de interfaz web (`/`) se despachan al contenedor frontend en el puerto 4173. Las peticiones a la API (`/api/*`) se reescriben y despachan al backend en el puerto 3001.
3. **Seguridad y Negocio:** El backend valida los tokens JWT, ejecuta las reglas de negocio, y accede al contenedor PostgreSQL mediante un connection pool optimizado.

3. Stack Tecnológico Detallado

3.1. Frontend (Capa de Presentación)

- **React 18.3 & TypeScript:** Interfaz de usuario reactiva, fuertemente tipada para prevenir errores en tiempo de ejecución.
- **Vite 5:** Empaquetador y servidor de desarrollo ultra-rápido, con árbol de dependencias optimizado y minificación de assets.
- **Progressive Web App (PWA):** Implementado mediante `vite-plugin-pwa` y Workbox, permitiendo la instalación en la pantalla de inicio de dispositivos móviles (Android / iOS) con apariencia de aplicación nativa.
- **Lucide Icons & SVG Vectoriales:** Íconos estilizados de alta resolución, complementados con componentes vectoriales SVG custom para la silueta frontal de vehículos y el pin oficial de Google Maps.
- **Vanilla CSS Moderno:** Arquitectura CSS con variables (`--primary: #FF6000` , `--ink: #0F172A`), sistema de pills, tarjetas de KPIs y diseño 100% responsivo para móviles y escritorios.
- **Compresión de Imágenes en Cliente:** Reducción automática de tamaño de fotos de odómetro mediante Canvas antes de su envío a la base de datos, optimizando el consumo de ancho de banda móvil.

3.2. Backend (Capa de Servicios y Lógica de Negocio)

- **Node.js 20 LTS & Express 4:** Servidor REST asíncrono, con manejo robusto de middleware de autorización y ruteo estructurado por módulos.
- **node-postgres (`pg`):** Driver nativo de PostgreSQL con Connection Pooling (`max: 20` , `idleTimeoutMillis: 30000`).
- **JWT (JSON Web Tokens):** Cifrado de credenciales stateless mediante firmas HMAC-SHA256, verificando identidad y rol en cada transacción.
- **Bcryptjs:** Algoritmo de hashing irreversible con salt rounds de 10 para todas las contraseñas de usuarios.
- **ExcelJS:** Generación en memoria de hojas de cálculo de Microsoft Excel (.xlsx) con estilos corporativos BICSA, cabeceras institucionales, formateo de números en Guaraníes y múltiples solapas analíticas.

3.3. Base de Datos (Persistencia)

- **PostgreSQL 16 Alpine:** Motor relacional de alto rendimiento y bajo consumo de memoria RAM.
- **Extensión pgcrypto:** Generación nativa de identificadores UUIDv4 mediante `gen_random_uuid()` .
- **Tipos ENUM Nativos:** Restricción estricta en base de datos para roles (`user_role`), etapas del embudo comercial (`funnel_stage`) y resultados (`visit_result`).
- **Columnas Generadas (STORED):** Cálculo automático e infalible de kilometraje total: `km_total INTEGER GENERATED ALWAYS AS (km_end - km_start) STORED` .
- **Triggers PL/pgSQL:** Actualización automática de marcas temporales `updated_at` en todas las tablas principales.

4. Composición del Despliegue Docker Compose

El archivo `docker-compose.yml` orquesta la infraestructura completa. A continuación se desglosan los 4 servicios:

Servicio	Imagen / Build	Puerto Host	Puerto Contenedor	Volúmenes y Dependencias
postgres	`postgres:16-alpine`	5434	5432	Volumen: `bicsa_comercial_postgres_data`. Scripts init en `/database/`
backend	Dockerfile (Node.js 20)	No expuesto	3001	Depende de: `postgres` (condición: healthy). Código en `/app`
frontend	Dockerfile (Node 20 + Serve)	No expuesto	4173	Build de producción minificado y servido con `serve -s dist`
nginx	`nginx:alpine`	8088	80	Monta `./nginx/default.conf`. Depende de: `frontend` y `backend`

Aislamiento de Seguridad en Puertos

Ni el frontend (4173) ni el backend (3001) exponen puertos al exterior del servidor. Únicamente Nginx recibe tráfico externo a través del puerto configurado (8088 en host o 80/443 con proxy inverso global), garantizando que nadie pueda evadir las cabeceras de seguridad y autenticación.

5. Variables de Entorno y Configuración (.env)

El sistema se configura a través del archivo `.env` en la raíz del repositorio. Este archivo nunca debe compartirse ni subirse al repositorio público de Git.

Variable	Ejemplo en Producción	Descripción y Propósito
POSTGRES_DB	bicsa_db	Nombre de la base de datos PostgreSQL
POSTGRES_USER	bicsa_user	Usuario propietario con permisos de lectura y escritura
POSTGRES_PASSWORD	[Clave-Secreta-Alta-Seguridad]	Contraseña del usuario de base de datos
POSTGRES_HOST_PORT	5434	Puerto en el host para conexiones de mantenimiento o pgAdmin
NODE_ENV	production	Modo de ejecución de Node.js (optimiza rendimiento)
PORT	3001	Puerto interno del servidor Express
JWT_SECRET	[Cadena-Hex-256-Bits]	Clave privada para firmar y validar tokens criptográficos de sesión
JWT_EXPIRES_IN	8h	Tiempo de validez de la sesión antes de requerir re-login
DATABASE_URL	postgresql://bicsa_user:pass@postgres:5432/bicsa_db	Cadena de conexión interna entre contenedores Docker
VITE_API_URL	/api	Ruta base del cliente para despachar peticiones AJAX al proxy
PORT_HTTP	8088	Puerto HTTP expuesto en el servidor anfitrión

6. Modelo de Base de Datos PostgreSQL 16

El esquema relacional fue diseñado bajo estándares de **tercera forma normal (3NF)** con integridad referencial estricta y llaves foráneas con eliminación restringida (`ON DELETE RESTRICT`) para proteger la trazabilidad histórica de visitas y salidas.

Diccionario de Tablas Principales:

Tabla	Campos Clave	Descripción y Reglas de Negocio
users	`id, email, password_hash, full_name, role, is_active, photo_odometer_enabled, last_login_at`	Almacena las cuentas de ejecutivos, gerentes y administradores. Controla el estado activo/inactivo y la obligatoriedad de fotos.
companies	`id, name, normalized_name, commercial_status, department, city, zone, maps_url, notes`	Directorio corporativo de clientes y prospectos. `normalized_name` cuenta con índice único para evitar duplicados.
work_days	`id, user_id, date, km_start, km_end, km_total, status, mobility_type, fuel_amount_gs, platform_trips`	Jornadas diarias de trabajo. Valida que el odómetro final sea mayor o igual al inicial mediante restricción `CHECK`.
visits	`id, work_day_id, user_id, company_id, date, reason, funnel_stage, result, transport_cost_gs, total_cost_gs`	Visitas comerciales asociadas a una jornada y una empresa. Registra costos de plataformas, combustible prorrateado y viáticos.
audit_log	`id, action, entity_type, entity_id, target_user_id, performed_by, details, created_at`	Traza de auditoría inmutable. Registra cualquier modificación correctiva hecha por administradores en jornadas cerradas.

7. Infraestructura en la Nube (Oracle Cloud OCI)

La aplicación se encuentra alojada en una instancia de cómputo en la nube de **Oracle Cloud Infrastructure (OCI)** bajo el sistema operativo **Ubuntu Linux LTS**. Para garantizar la disponibilidad continua, deben configurarse las siguientes políticas de red:

7.1. Reglas de Entrada (Ingress Rules) en OCI Virtual Cloud Network (VCN)

En la consola web de Oracle Cloud, navegue a *Networking* > *Virtual Cloud Networks* > *Security Lists* > *Default Security List* y verifique las siguientes reglas de ingreso:

Protocolo	CIDR Origen	Puerto Destino	Propósito Operativo
TCP	`0.0.0.0/0`	**22**	Acceso administrativo SSH con llave criptográfica
TCP	`0.0.0.0/0`	**80**	Tráfico web HTTP y validación de desafíos Let's Encrypt
TCP	`0.0.0.0/0`	**443**	Tráfico web seguro HTTPS encriptado
TCP	`0.0.0.0/0`	**8088**	Puerto alternativo directo de la aplicación BICSA Nginx

7.2. Configuración del Cortafuegos Interno de Ubuntu (iptables / ufw)

Las imágenes de Ubuntu en Oracle Cloud incluyen reglas por defecto en `iptables` que bloquean puertos no estándar. Para habilitar los puertos necesarios, ejecute en la consola del servidor:

```
# Abrir puertos en el cortafuegos de Ubuntu
sudo iptables -I INPUT 6 -m state --state NEW -p tcp --dport 80 -j ACCEPT
sudo iptables -I INPUT 6 -m state --state NEW -p tcp --dport 443 -j ACCEPT
sudo iptables -I INPUT 6 -m state --state NEW -p tcp --dport 8088 -j ACCEPT
sudo netfilter-persistent save
```

8. Configuración de Dominio y Registros DNS

Para que los usuarios puedan acceder al sistema mediante una dirección web amigable (en lugar de recordar la dirección IP numérica), se configura la zona DNS:

1. **Obtención de la IP Pública:** En la consola de Oracle Cloud, identifique la *Public IPv4 Address* asignada a su máquina virtual (ejemplo: `129.151.108.50`). Se recomienda que sea una IP estática (Reserved Public IP) para evitar cambios accidentales al reiniciar.

2. **Creación del Registro Tipo A en el Proveedor DNS:** En el panel de su proveedor de dominio (NIC.py, Cloudflare, DuckDNS o GoDaddy), añada:

Tipo de Registro	Nombre / Host	Valor / Destino	TTL
A	`comercial` (o subdominio asignado)	`[IP Pública de tu Oracle Cloud]`	300 seg (5 min) o Automático

3. **Verificación de Propagación:** Compruebe la correcta resolución ejecutando en su terminal local:

```
nslookup bicsa-gestion-comercial.duckdns.org
# o bien:
dig +short comercial.bicsa.com.py
```

9. Certificados de Seguridad SSL/TLS y Terminación HTTPS

La plataforma requiere HTTPS obligatorio para proteger las contraseñas de los usuarios, los datos geográficos de visitas y para permitir que los navegadores móviles autoricen el funcionamiento de la cámara (foto de odómetro) y la geolocalización.

9.1. Opción 1: Certificado Automático con Traefik / Coolify (Integrado)

El archivo `docker-compose.yml` ya contiene las etiquetas (`labels`) para la emisión automática de certificados de **Let's Encrypt**:

```
labels:
  - "traefik.enable=true"
  - "traefik.docker.network=coolify"
  - "traefik.http.routers.bicsa-comercial.entrypoints=https"
  - "traefik.http.routers.bicsa-comercial.rule=Host(`bicsa-gestion-comercial.duckdns.org`)"
  - "traefik.http.routers.bicsa-comercial.tls.certresolver=letsencrypt"
  - "traefik.http.services.bicsa-comercial.loadbalancer.server.port=80"
```

Bajo este esquema, el proxy maestro Coolify/Traefik valida el dominio contra los servidores de Let's Encrypt y renueva el certificado cada 60 días de manera 100% desatendida.

9.2. Opción 2: Certificado Standalone con Certbot en el Host

Si se administra Nginx de forma nativa sin Traefik, se puede generar y renovar el certificado SSL mediante Certbot en el servidor Ubuntu:

```
# Instalar Certbot oficial
sudo apt update && sudo apt install -y certbot python3-certbot-nginx

# Solicitar certificado para el dominio oficial
sudo certbot certonly --standalone -d comercial.bicsa.com.py -m soporte@bicsa.com.py --agree-tos

# Renovar automáticamente mediante tarea programada (cron)
sudo crontab -e
# Agregar línea para renovar todos los lunes a las 03:00 AM:
0 3 * * 1 certbot renew --quiet && docker compose -f /home/ubuntu/SISTEMA_COMERCIAL_BICSA/docker-compose.yml r
```

10. Mecanismos de Autenticación, JWT y Auto-Cierre a Medianoche

10.1. Tokens JWT Stateless

Al iniciar sesión exitosamente, el backend emite un token JWT codificado con el algoritmo `HS256` que contiene el `userId`, `email`, `role` y `full_name`. El cliente web almacena este token de manera segura en el almacenamiento

local y lo adjunta en la cabecera HTTP `Authorization: Bearer [TOKEN]` en cada petición.

10.2. Política de Auto-Cierre de Sesión a las 00:00 hs

> **Regla de Seguridad Operativa**
>
> Por directiva institucional de auditoría comercial, ningún ejecutivo ni usuario puede mantener una sesión abierta que trascienda al día siguiente. Al llegar las 00:00:00 de cada noche, cualquier sesión activa se invalida automáticamente.

Esta funcionalidad está programada en el módulo `AuthContext.tsx` del frontend:

```
// Cálculo milimétrico de la próxima medianoche en tiempo local
const now = new Date();
const nextMidnight = new Date(now.getFullYear(), now.getMonth(), now.getDate() + 1, 0, 0, 1);
const msUntilMidnight = nextMidnight.getTime() - now.getTime();

// Programación del temporizador que forzará el cierre de sesión al cambiar de día
const midnightTimer = setTimeout(() => {
  logout();
  window.location.href = '/login?expired=midnight';
}, msUntilMidnight);
```

11. Estrategia de Copias de Seguridad (Backups y Restauración)

La información comercial y de visitas es un activo crítico de BICSA. Se define una política de respaldos automatizados con rotación de 30 días.

11.1. Script de Backup Diario Automatizado

Cree el archivo `/home/ubuntu/backup_bicsa.sh` en su servidor:

```
#!/bin/bash
BACKUP_DIR="/home/ubuntu/backups_bicsa"
mkdir -p $BACKUP_DIR
FECHA=$(date +"%Y%m%d_%H%M%S")
ARCHIVO="$BACKUP_DIR/bicsa_backup_$FECHA.sql.gz"

# Ejecutar volcado en caliente desde el contenedor PostgreSQL
docker exec bicsa_comercial_postgres pg_dump -U bicsa_user -d bicsa_db | gzip > "$ARCHIVO"

# Eliminar respaldos más antiguos a 30 días para no saturar el disco
find $BACKUP_DIR -name "bicsa_backup_*.sql.gz" -mtime +30 -delete

echo "Backup completado exitosamente: $ARCHIVO"
```

11.2. Programación en Crontab

Ejecutar `crontab -e` y programar el volcado diario a las 02:00 AM:

```
0 2 * * * /bin/bash /home/ubuntu/backup_bicsa.sh >> /home/ubuntu/backup.log 2>&1
```

11.3. Procedimiento de Restauración de Desastres

En caso de requerir restaurar una copia de seguridad en un servidor nuevo o tras un incidente:

```
# 1. Asegurarse de que el contenedor de base de datos esté corriendo
docker compose up -d postgres

# 2. Descomprimir e inyectar el backup en la base de datos limpia
gunzip -c /home/ubuntu/backups_bicsa/bicsa_backup_20261023_020000.sql.gz | docker exec -i bicsa_comercial_post
```

12. Comandos Operativos de Despliegue y Mantenimiento

Acción Operativa	Comando de Terminal a Ejecutar

Actualizar código y frontend a última versión	`cd ~/SISTEMA_COMERCIAL_BICSA`
	`git pull origin main`
	`docker compose build frontend`
	`docker compose up -d --force-recreate frontend`
	`docker compose restart nginx`

Verificar estado de los 4 contenedores	`docker compose ps`
Inspeccionar logs de errores del backend en vivo	`docker compose logs -f --tail=100 backend`
Inspeccionar peticiones web en Nginx	`docker compose logs -f --tail=100 nginx`
Reiniciar el sistema completo	`docker compose restart`
Detener y levantar desde cero	`docker compose down && docker compose up -d`

Soporte y Contacto Técnico

Para reportar anomalías, auditorías de seguridad o solicitudes de cambios en infraestructura, comuníquese con el equipo de soporte técnico institucional de BICSA S.A.